Clash of Codes

Real-Time Competitive Coding & Strategy Game

Game Design Document

&

4-Week ASP.NET Core + Blazor Build Guide

Tech Stack

ASP.NET Core 8 • Blazor Server
SQL Server + EF Core • SignalR
Monaco Editor • Judge0 API
Azure App Service

Version 1.0 • 2026

This document combines the original Game Design Document with recommended design improvements and a complete day-by-day development roadmap for first-time web developers.

Contents

Abstract **2**

1 The Core Game Loop & Resource Economy **3**
1.1 Battle Elixir (BE) . . . 3
1.2 Rank Points (RP) and the Conversion Mechanic . . . 3

2 Match Structure & Victory Conditions **4**
2.1 The Three-Track Node Setup . . . 4
2.2 Node Workflow Structure . . . 4

3 The Spellbook Mechanics **4**
3.1 Offensive Spells (Attacks) . . . 4
3.2 Defensive Spells (Counter-Play) . . . 5

4 Matchmaking Models & Environments **5**
4.1 Ranked Ladder . . . 5
4.2 Custom Lobbies . . . 6
4.3 Syntax Stages (Arenas) . . . 6

5 Recommended Design Improvements **7**
5.1 Spell Cooldown System . . . 7
5.2 Active Debuff Cap . . . 7
5.3 Surrender & Disconnect Handling . . . 7
5.4 Define $\delta = 5$ . . . 7
5.5 Spell Notifications (Toasts) . . . 7

6 Technology Stack **8**

7 4-Week Day-by-Day Build Guide **9**
7.1 Week 1 — Foundations . . . 9
7.2 Week 2 — Core Game Loop . . . 10
7.3 Week 3 — Spells & Real-Time Multiplayer . . . 12
7.4 Week 4 — Polish & Deployment . . . 13

A Key Commands Reference **15**

B Recommended Learning Resources **15**

C Suggested Database Schema **16**

Abstract

Clash of Codes is a multiplayer competitive programming game that adapts the high-stakes, real-time resource management of the Clash Royale format into an algorithmic battleground. Players match with opponents globally or challenge friends in private rooms, balancing fast-paced code composition, multiple-choice questions, and disruptive environmental abilities called **Spells**. This document outlines the fundamental loop, economic resource models, combat mechanics, and matchmaking criteria, together with recommended design improvements and a complete developer build roadmap.

1 The Core Game Loop & Resource Economy

The gameplay ecosystem transforms isolated technical challenges into a fluid strategy loop. The engine tracks two distinct resources: **Battle Elixir (BE)** and **Rank Points (RP)**.

1.1 Battle Elixir (BE)

Battle Elixir is the volatile, real-time resource used exclusively during an active match to deploy tactical spells.

- **Capacity Limits:** All players start a match with 0 BE. Maximum capacity is hard-capped at **10 BE**.
- **Passive Generation:** Players regenerate **1 BE every 10 seconds**. In the final 60 seconds, **Double Elixir mode** activates, doubling the rate to **2 BE every 10 seconds**.
- **Active Generation:** Milestone bonuses reward efficient play:

Milestone	BE Bonus
Successful Compilation	+1 BE
Flawless First-Submission Run	+3 BE
Correct MCQ Answer	+1 BE

1.2 Rank Points (RP) and the Conversion Mechanic

Rank Points dictate a user's global ladder standing. The game implements a **Greed vs. Power Economy**:

$$\text{Final RP Awarded} = \text{Match Base Victory RP} + \Big(\text{Unspent BE Remaining} \times \delta\Big)$$

Where δ is the static scaling multiplier (**recommended:** $\delta = 5$). This forces a critical choice: burn Elixir on spells to guarantee a win, or conserve it to maximise RP gains on victory.

After 3 consecutive losses, the RP penalty for the next match is halved. This prevents new players from getting trapped in demotion spirals and is standard in competitive ladder games.

2 Match Structure & Victory Conditions

Matches display the player's active IDE alongside a real-time progress tracker showing the opponent's test case pass rates.

2.1 The Three-Track Node Setup

A standard map consists of three algorithmic tracks:

1. **Left Track (Easy):** Yields 1 Crown on completion.
2. **Right Track (Medium):** Yields 1 Crown on completion.
3. **Centre Track (Hard):** The King's Tower. Completing this instantly triggers a **3-Crown absolute victory**, overriding all other conditions.

2.2 Node Workflow Structure

Every Problem Node follows a two-stage sequential architecture:

Stage I: Code Composition (pass public test cases) $\longrightarrow$ **Stage II:** MCQ Stream (3–5 questions)

Wrong code submissions deduct flat point values; incorrect MCQ answers apply negative point penalties.

3 The Spellbook Mechanics

Before entering matchmaking, players assemble a loadout of exactly **4 Spells** (2 Offensive + 2 Defensive).

The full list contains 17 spells. Building and balancing all of them while learning Blazor in 4 weeks is unrealistic. **Launch with 6 spells only** (3 offensive + 3 defensive), then add more after playtesting.

Recommended first set: Fog of War, Time Warp, MCQ Barrage + Lint Shield, Garbage Collector, Ctrl+Z Rollback

3.1 Offensive Spells (Attacks)

Spell Name	BE Cost	Impact
Fog of War	2	Hides compiler line numbers for 45s; opponent sees only "Compilation Failed" with no details.
Time Warp	3	Distorts the opponent's match timer to spin visually at double speed for 30s.
Chaff	4	Obfuscates 50% of sample input/output values on the opponent's interface.

(continued)

Spell Name	BE Cost	Impact
EMP (Flicker)	4	Scrambles editor line numbers and flashes the screen dynamically for 15s.
Syntax Poison	4	Switches opponent's IDE theme to an ultra-low contrast colour scheme for 30s.
MCQ Barrage	5	Force-injects 2 high-difficulty MCQ items into the opponent's validation queue.
The Typo Curse	5	For 30s, each semicolon (`;`) has a 30% chance of mapping to a Greek question mark, breaking compilation.
Bracket Bandit	6	Instantly deletes all trailing closing brackets (`}`, `]`, `)`) from the opponent's editor.
Memory Leak	6	Caps execution runtime for 45s, causing valid code to fail with an artificial TLE.
Code Swap	8	Replaces the opponent's current problem with a new one of equivalent difficulty, resetting all progress.

3.2 Defensive Spells (Counter-Play)

Spell Name	BE Cost	Impact
Clock Sync	1	Instantly re-aligns the local timer with the true server timestamp, removing Time Warp.
Lint Shield	2	Makes the local IDE immune to flickering, theme changes, and typo curses for 60s.
Debugger Sight	3	Permanently reveals 2 hidden edge-case inputs for the active problem node.
Sanitize Input	3	Instantly purges active UI obfuscations caused by incoming Chaff spells.
Ctrl+Z (Roll-back)	4	Activates a 3s reactive window; if a Code Swap lands, this restores the previous code buffer.
Firewall	4	Prevents a selected Problem Track from being altered or swapped for 90s.
Garbage Collector	5	Purges all active debuffs, visual distortions, and status impairments from the player's UI.

4 Matchmaking Models & Environments

4.1 Ranked Ladder

The public matchmaking algorithm pairs players using an Elo-based standard on current global RP. Players within ±**200 RP** are eligible for pairing. Questions are selected

randomly across sub-disciplines (Arrays, Dynamic Programming, Graph Theory) matched to the arena tier.

4.2 Custom Lobbies

Users can bypass public queues:

- **Room Codes:** A 6-character alphanumeric key for peer connection.
- **Direct Invites:** Push notification via the opponent's User ID.
- **Configuration:** Host can filter topics, cap difficulty, or set Elixir scaling (e.g., Infinite 7x Elixir mode).

4.3 Syntax Stages (Arenas)

Arena	Name & RP Range	Problem Themes
1	Spaghetti Junction (0–499)	Primitive ops, arrays, string parsing
2	Parse Pit (500–999)	Sorting, two-pointer, prefix sums
3	Stack Shack (1000–1499)	Stacks, queues, sliding window
4	Heap Hollow (1500–1999)	Binary search, heaps, greedy
5	The Stack Overflow (2000–2499)	Recursive trees, heavy graph algorithms
6	Pointer Purgatory (2500–2999)	Custom objects, DFS/BFS mastery
7	Bitwise Badlands (3000–3499)	Bit manipulation, advanced graphs
8	Complexity Citadel (3500–3999)	Dynamic programming (medium-hard)
9	Recursion Realm (4000–4999)	Advanced DP, segment trees
10	Kernel Palace (5000+)	Systems architecture, multi-variable scheduling

Each new arena unlocks one additional spell in the Spellbook, giving players a progression incentive beyond prestige.

5 Recommended Design Improvements

5.1 Spell Cooldown System

Without cooldowns, a player with 10 BE could immediately fire *Bracket Bandit* (6 BE) and *Code Swap* (8 BE) in rapid succession. Add a **30–60 second cooldown per spell slot** after casting.

5.2 Active Debuff Cap

Stacking Fog of War + Syntax Poison + EMP simultaneously is unplayable. Rule:

> A player may have at most **2 active offensive debuffs** on the opponent at any time. A third spell replaces the oldest active debuff.

5.3 Surrender & Disconnect Handling

If a player is disconnected for **60 or more seconds**, they automatically forfeit and the opponent wins with a partial RP bonus.

5.4 Define $\delta = 5$

With base victory RP at 30–50 points, $\delta = 5$ gives a maximum bonus of +50 RP for a full unspent elixir pool — meaningful but not dominant. Adjust after playtesting.

5.5 Spell Notifications (Toasts)

When a spell hits, display a clear notification, for example:

"Opponent cast Fog of War — compiler messages hidden for 45s"

Without this, spell effects appear as random bugs rather than intentional gameplay.

6 Technology Stack

Technology	Role in Clash of Codes
ASP.NET Core 8 (Blazor Server)	Backend and UI in one. Blazor Server runs C# on the server and pushes UI updates over a persistent connection. No JavaScript needed for the main UI.
SQL Server Express + EF Core	Free local database. EF Core generates SQL from C# classes automatically via migrations.
ASP.NET Core SignalR	Real-time communication for opponent progress, spell broadcasts, timer sync, and Double Elixir events.
Monaco Editor (JS Interop)	The VS Code editor embedded in the browser via CDN. Controlled through Blazor's `IJSRuntime`.
Judge0 CE API	Free cloud code execution. 100 submissions/day on the free tier, sufficient for development. Self-hostable for production.
Azure App Service	Deployment target with SignalR and Blazor Server support on the free tier.

1. **.NET 8 SDK** — https://dotnet.microsoft.com/download/dotnet/8.0
2. **VS Code** + C# Dev Kit extension (free)
3. **SQL Server Express** + Azure Data Studio
4. **Git** — create a GitHub repo on Day 1 and push every day
5. **Blazor docs** — https://learn.microsoft.com/en-us/aspnet/core/blazor/

7 4-Week Day-by-Day Build Guide

Week	Theme	Goal at end of week
1	Foundations	Auth, database, basic Blazor UI, profile page
2	Core Game Loop	Code editor, Judge0, MCQ, BE economy, solo play
3	Multiplayer	SignalR real-time match, spells, matchmaking queue
4	Polish & Deploy	Ranked ladder, private lobbies, UI polish, Azure deploy

7.1 Week 1 — Foundations

Components, `@page` routing, `@inject`, `EditForm`, EF Core migrations, ASP.NET Identity, `AuthorizeView`.

Days 1–2: Project scaffold & Blazor introduction

Day 1 **Create the solution**

Run `dotnet new blazorserver -o ClashOfCodes`. Explore the default template — understand what `_Host.cshtml`, `App.razor`, and the `Pages/` folder do. Push to GitHub. Key concept: a `.razor` file is HTML + C# in one file.

Day 2 **Layout, navigation, shared components**

Edit `MainLayout.razor` to add a top navigation bar. Create `Shared/NavBar.razor`. Learn `@inject` and `NavigationManager`. Add placeholder pages: Home, Login, Game. Learn `@page` routing.

Days 3–4: Database & Entity Framework Core

Day 3 **Design and create the database schema**

Install EF Core: `dotnet add package Microsoft.EntityFrameworkCore.SqlServer`. Create models: `User`, `Problem`, `Match`, `MatchPlayer`. Write `AppDbContext`. Run first migration: `dotnet ef migrations add Init`.

Day 4 Seed problems and verify data

Write a `DbSeeder` inserting 5 easy, 5 medium, and 3 hard problems with MCQ questions. Open Azure Data Studio and browse your tables. Build a Blazor admin page listing problems with `@foreach` — practice injecting `AppDbContext` into a component.

Days 5–7: Authentication & user profiles

Day 5 ASP.NET Identity setup

Add `Microsoft.AspNetCore.Identity.EntityFrameworkCore`. Extend `IdentityUser` with: `RankPoints`, `CurrentArena`, `BattleElixir`, `SelectedSpells`. Run migration. Wire up cookie authentication in `Program.cs`.

Day 6 Register and login pages in Blazor

Build `Register.razor` and `Login.razor` using `EditForm` and `DataAnnotationsValidator`. Call `UserManager` and `SignInManager`. Learn `AuthenticationStateProvider` and `<AuthorizeView>` for conditional nav items.

Day 7 Profile page and spellbook selector

Build `Profile.razor`: show username, RP, arena badge, wins/losses. Add a Spellbook selector displaying all spells as cards; let the user pick 2+2. Save selection to the database. Use `[Authorize]` to protect the page.

7.2 Week 2 — Core Game Loop

JS Interop with `IJSRuntime`, `HttpClient` and DI, `StateHasChanged()`, `EventCallback`, server-side timers.

Days 8–9: Code editor + Judge0 API

Day 8 Embed Monaco Editor

Load Monaco Editor via CDN using Blazor JS Interop. Write a `CodeEditor.razor` component with a `GetCode()` method. This teaches `IJSRuntime` — a critical skill for the spell system.

Day 9 Code submission with Judge0

Sign up at `https://judge0.com` (free tier). Create a `JudgeService` that sends code and test cases via `HttpClient`, polls for results, and returns pass/fail per test case. Register it in `Program.cs` via DI.

Days 10–11: Problem node, MCQ, and scoring

Day 10 MCQ stage after code passes

After all public test cases pass, transition to an MCQ stream (3–5 questions from the database). Build `McqPanel.razor`. Correct answer: +1 BE and advance. Wrong answer: score penalty. Learn `EventCallback` to emit results to the parent component.

Day 11 BE economy and crown system

Implement BE generation: passive timer via `System.Threading.Timer`; active bonuses on compile/flawless/MCQ events. Track 3 crowns per match. Build a crown indicator component. Centre node completion = instant 3-crown win.

Days 12–14: Solo practice mode (full loop)

Day 12 Full single-player game screen

Build `GamePage.razor`: 3-track layout, problem display, code editor, submit button, crown display, BE bar, 5-minute countdown. Wire all components together. Play the full solo loop and fix bugs before adding multiplayer.

Day 13 Match result screen and RP calculation

Implement: FinalRP = BaseRP + (UnspentBE × 5). Build a post-match screen showing crowns, RP gained, new total, and arena progress. Save the result to a `MatchHistory` table.

Day 14 Buffer and catch-up day

Review Week 2. Fix UI polish. Add 5 more seeded problems. Read the SignalR documentation before Week 3: `https://learn.microsoft.com/en-us/aspnet/core/signalr/introduction`.

7.3 Week 3 — Spells & Real-Time Multiplayer

SignalR hubs, `HubConnection`, `On`/`SendAsync`, in-memory match state with `ConcurrentDictionary`, `BackgroundService` for matchmaking.

Days 15–17: SignalR real-time multiplayer

Day 15 Blazor + SignalR introduction

Add `Microsoft.AspNetCore.SignalR`. Create `GameHub.cs`. Simple test: two browser tabs — type in one, the opponent's progress bar updates in the other. Learn `HubConnection`, `On`, and `SendAsync`. This is the hardest concept of the project — take your time.

Day 16 Match state sync: test cases and crowns

Broadcast opponent test-case pass percentage and crown captures via SignalR. Store match state in a server-side `ConcurrentDictionary<string, MatchState>` — no database writes during the match, only on completion.

Day 17 Double Elixir mode and match end detection

At T-60s, broadcast `DoubleElixirStarted`; both clients switch BE rate to 2/10s. Detect match end: 3-crown instant win or most crowns at timer zero. Tie-breaker: most test cases passed. Broadcast winner, save result to database.

Days 18–20: Spellbook system

Day 18 Offensive spells — UI disruption

Implement *Fog of War* (hide compiler messages via JS), *Time Warp* (CSS animation distorts timer display), *Syntax Poison* (inject a low-contrast theme class into Monaco via JS Interop). Each spell broadcasts via SignalR; the opponent's client applies the effect.

Day 19 Offensive spells — code disruption

Implement *MCQ Barrage* (push 2 extra items into opponent's server-side MCQ queue), *Memory Leak* (reduce time limit in Judge0 call for opponent's next submission), *Bracket Bandit* (JS Interop deletes trailing brackets from Monaco buffer).

Day 20 Defensive spells and spell hotbar UI

Implement *Lint Shield* (60s immunity flag), *Garbage Collector* (clear all debuff flags), *Ctrl+Z Rollback* (3s window with saved code snapshot). Build the spell hotbar: 4 cards with name, BE cost, and cooldown ring animation.

Day 21: Matchmaking queue

Day 21 Ranked queue and match creation

Build a `MatchmakingQueue` as a `BackgroundService`. When a user clicks "Find Match" they join the queue. Every 5 seconds, pair two players within ±200 RP. Create a Match record, assign 3 random problems (1 easy, 1 medium, 1 hard), redirect both players to `/game/{matchId}`.

7.4 Week 4 — Polish & Deployment

`IQueryable` for efficient DB queries, CSS Grid advanced layout, Azure App Service deployment, environment variables for secrets, end-to-end testing.

Days 22–23: Ranked ladder, arenas, leaderboard

Day 22 Arena promotion system

Map RP ranges to the 10 arena tiers. On match end, check if RP crosses a threshold and trigger a promotion animation. Store the current arena in the User record. Filter problem difficulty by arena in matchmaking.

Day 23 Global leaderboard page

Build `Leaderboard.razor`: query top 50 users by RP, show rank, username, arena badge, RP, and win rate. Add pagination. Add a "Your rank" card that always shows the current user's position even if outside the top 50.

Days 24–25: Private lobbies and custom rooms

Day 24 Room code generation

Build `CreateRoom.razor`: generate a 6-character code stored in a `Rooms` table. Host configures topic filter, difficulty cap, and Elixir mode. Redirect to the standard game flow on start.

Day 25 Lobby waiting room UI

Build a waiting room: show both players with avatar initials, a "Ready" button, live SignalR chat, and the room config summary. Host starts once both are ready. Add a share button that copies the room link to the clipboard.

Days 26–27: Visual design pass

Day 26 Global styling

Choose a consistent dark colour theme using CSS custom properties. Add entrance animations for spell cards and crown captures. Design arena badges as inline SVG. Add a pulse animation to the BE bar during Double Elixir mode.

Day 27 Game screen layout refinement

Use CSS Grid: left panel for problem and editor, right panel for opponent progress and spell hotbar. Add a collapsible problem statement. Test on 1280px and 1920px. Add keyboard shortcuts (e.g., Ctrl+Enter to submit code).

Days 28–30: Testing and deployment

Day 28 End-to-end testing with two accounts

Open two browsers (Chrome and Firefox). Register two accounts. Play a full ranked match from queue to result. Cast spells at each other. Verify all BE and RP calculations. Log every bug as a GitHub issue.

Day 29 Bug fixes and error handling

Fix all bugs from Day 28. Add proper error pages (404, 500). Add loading spinners. Handle Judge0 API timeouts gracefully: show a "Submission timed out" message instead of crashing.

Day 30 Deploy to Azure App Service

Create a free Azure account. Deploy via VS Code Azure extension or `dotnet publish` + Azure CLI. Set environment variables for the connection string and Judge0 API key. Share the URL with friends and play.

A Key Commands Reference

```
# Create a new Blazor Server project
dotnet new blazorserver -o ClashOfCodes

# Add Entity Framework Core packages
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Tools

# Create and apply a migration
dotnet ef migrations add <MigrationName>
dotnet ef database update

# Add ASP.NET Identity
dotnet add package Microsoft.AspNetCore.Identity.
   EntityFrameworkCore

# Run the project locally
dotnet run

# Publish for Azure deployment
dotnet publish -c Release -o ./publish
```

B Recommended Learning Resources

- Blazor documentation: https://learn.microsoft.com/en-us/aspnet/core/blazor/
- SignalR with Blazor tutorial: https://learn.microsoft.com/en-us/aspnet/core/tutorials/signalr-blazor
- EF Core getting started: https://learn.microsoft.com/en-us/ef/core/get-started/
- Judge0 CE API: https://judge0.com
- Monaco Editor: https://microsoft.github.io/monaco-editor/
- Azure App Service free tier: https://azure.microsoft.com/en-us/products/app-service/

C Suggested Database Schema

Table	Key Fields
Users	Id, Username, Email, PasswordHash, RankPoints, CurrentArena, Wins, Losses, SelectedSpells (JSON)
Problems	Id, Title, Description, Difficulty (1/2/3), Topic, TestCasesJson, HiddenTestCasesJson
McqQuestions	Id, ProblemId (FK), QuestionText, OptionsJson, CorrectIndex
Matches	Id, CreatedAt, EndedAt, WinnerId (FK), IsRanked, RoomCode
MatchPlayers	MatchId (FK), UserId (FK), CrownsEarned, FinalBE, RpChange, SpellsUsedJson
Rooms	Id, Code, HostUserId (FK), ConfigJson, Status (Waiting/Active/-Closed)